Министерство образования Республики Беларусь

Учреждение образования

Белорусский Государственный университет информатики и радиоэлектроники

Инженерно-экономический факультет

Кафедра экономической информатики

Отчет

по лабораторной работе № 2

на тему «Детализация интерфейсов и планирование разработки системы»

Выполнила: Абу Халава И.М.
студент группы 377901

Проверил: Сильванович Ю.В.

Минск 2026

Введение

Лабораторная работа продолжает архитектурную модель, разработанную в ЛР1. На этом этапе границы сервисов не меняются: система по-прежнему разделена на Identity Service, Master Data Service, Production Integration Service, Financial Integration Service, Costing Service, Profitability Analytics Service и Reporting Service. Основная задача состоит в детализации технических контрактов и структур данных, необходимых для реализации выбранных бизнес-сценариев.

В ЛР2 структуры баз данных рассматриваются уже на уровне конкретных таблиц, полей, типов и ключей. GraphQL описывает внешний контракт веб-клиента, gRPC используется для синхронного получения данных между сервисами, а Kafka фиксирует завершившиеся события и позволяет обновлять зависимые процессы без жесткой связи между компонентами.

Цель работы

Цель работы состоит в технической детализации распределенной системы анализа рентабельности продукции: определить структуры баз данных сервисов, спроектировать GraphQL API, описать gRPC-контракты и protobuf-модели, определить Kafka-события и их payload, а также подробно разобрать последовательность выполнения ключевых бизнес-сценариев.

1 Структура баз данных сервисов

При детализации сохраняется принцип Database per Service. Каждая таблица физически принадлежит одному сервису. Foreign key применяются только между таблицами одной базы данных. Идентификаторы сущностей из других сервисов хранятся как обычные UUID и проверяются через gRPC или события, поэтому межбазовые внешние ключи отсутствуют.

1.1 Identity Service - identity_db

USER

Учетная запись пользователя.

Поле	Тип	Ключ / ограничение	Назначение
user_id	UUID	PK	Идентификатор пользователя
email	VARCHAR(255)	UQ, NOT NULL	Адрес для входа
password_hash	VARCHAR(255)	NOT NULL	Хэш пароля
status	VARCHAR(30)	NOT NULL	Состояние учетной записи
created_at	TIMESTAMP	NOT NULL	Дата создания

ROLE

Справочник ролей доступа.

Поле	Тип	Ключ / ограничение	Назначение
role_id	UUID	PK	Идентификатор роли
name	VARCHAR(80)	UQ, NOT NULL	Системное имя роли
description	VARCHAR(255)		Описание роли

USER_ROLE

Связь M:N между пользователями и ролями.

Поле	Тип	Ключ / ограничение	Назначение
user_id	UUID	PK, FK -> USER	Пользователь
role_id	UUID	PK, FK -> ROLE	Роль

Поле	Тип	Ключ / ограничение	Назначение
assigned_at	TIMESTAMP	NOT NULL	Дата назначения

1.2 Master Data Service - master_data_db

PRODUCT_GROUP

Иерархический справочник групп продукции.

Поле	Тип	Ключ / ограничение	Назначение
group_id	UUID	PK	Идентификатор группы
parent_id	UUID	FK -> PRODUCT_GROUP, NULL	Родительская группа
code	VARCHAR(40)	UQ, NOT NULL	Код группы
name	VARCHAR(200)	NOT NULL	Наименование

PRODUCT

Карточка вида продукции.

Поле	Тип	Ключ / ограничение	Назначение
product_id	UUID	PK	Идентификатор продукции
group_id	UUID	FK -> PRODUCT_GROUP, NOT NULL	Группа продукции
code	VARCHAR(40)	UQ, NOT NULL	Внутренний код
name	VARCHAR(255)	NOT NULL	Наименование
unit	VARCHAR(20)	NOT NULL	Единица измерения
active	BOOLEAN	NOT NULL	Признак использования

MATERIAL

Справочник сырья и материалов.

Поле	Тип	Ключ / ограничение	Назначение
material_id	UUID	PK	Идентификатор материала
code	VARCHAR(40)	UQ, NOT NULL	Код материала
name	VARCHAR(255)	NOT NULL	Наименование
unit	VARCHAR(20)	NOT NULL	Единица измерения
standard_price	DECIMAL(18,2)		Нормативная цена

PRODUCT_MATERIAL

Нормативный состав продукта.

Поле	Тип	Ключ / ограничение	Назначение
product_id	UUID	PK, FK -> PRODUCT	Продукция
material_id	UUID	PK, FK -> MATERIAL	Материал
valid_from	DATE	PK	Начало действия нормы
standard_qty	DECIMAL(18,4)	NOT NULL	Количество по норме
valid_to	DATE	NULL	Окончание действия

COST_CENTER

Справочник центров затрат.

Поле	Тип	Ключ / ограничение	Назначение
cost_center_id	UUID	PK	Идентификатор центра
code	VARCHAR(40)	UQ, NOT NULL	Код подразделения
name	VARCHAR(255)	NOT NULL	Наименование
allocation_base	VARCHAR(40)	NOT NULL	База распределения

EXTERNAL_MAPPING

Сопоставление кодов внешних систем с внутренними идентификаторами.

Поле	Тип	Ключ / ограничение	Назначение
mapping_id	UUID	PK	Идентификатор сопоставления
source_system	VARCHAR(80)	NOT NULL	ERP, MES или финансовая система
entity_type	VARCHAR(40)	NOT NULL	PRODUCT, MATERIAL, COST_CENTER
external_code	VARCHAR(80)	NOT NULL	Код во внешней системе
internal_id	UUID	NOT NULL	Идентификатор объекта Master Data
status	VARCHAR(30)	NOT NULL	Состояние сопоставления
updated_at	TIMESTAMP	NOT NULL	Время изменения

1.3 Production Integration Service - production_db

PRODUCTION_ORDER

Факт выпуска продукции по производственному заказу.

Поле	Тип	Ключ / ограничение	Назначение
order_id	UUID	PK	Внутренний идентификатор
external_id	VARCHAR(80)	UQ, NOT NULL	Идентификатор в MES/ERP
product_id	UUID	NOT NULL	Идентификатор продукции из Master Data
period	DATE	NOT NULL	Отчетный период
quantity	DECIMAL(18,4)	NOT NULL	Фактический выпуск
status	VARCHAR(30)	NOT NULL	Состояние заказа

MATERIAL_CONSUMPTION

Фактическое списание материалов.

Поле	Тип	Ключ / ограничение	Назначение
consumption_id	UUID	PK	Идентификатор записи
order_id	UUID	FK -> PRODUCTION_ORDER	Производственный заказ
material_id	UUID	NOT NULL	Материал из Master Data
quantity	DECIMAL(18,4)	NOT NULL	Списанное количество
amount	DECIMAL(18,2)	NOT NULL	Фактическая стоимость
posted_at	TIMESTAMP	NOT NULL	Время проведения

LABOR_ENTRY

Трудовые затраты по производственному заказу.

Поле	Тип	Ключ / ограничение	Назначение
labor_entry_id	UUID	PK	Идентификатор записи
order_id	UUID	FK -> PRODUCTION_ORDER	Производственный заказ
cost_center_id	UUID	NOT NULL	Центр затрат из Master Data
hours	DECIMAL(12,2)	NOT NULL	Отработанные часы
amount	DECIMAL(18,2)	NOT NULL	Стоимость труда

IMPORT_RUN

Журнал загрузок производственных данных.

Поле	Тип	Ключ / ограничение	Назначение
import_run_id	UUID	PK	Идентификатор запуска
source_system	VARCHAR(80)	NOT NULL	Источник

Поле	Тип	Ключ / ограничение	Назначение
period	DATE	NOT NULL	Отчетный период
status	VARCHAR(30)	NOT NULL	RUNNING, COMPLETED, FAILED
records_loaded	INTEGER	NOT NULL	Успешно загружено
records_failed	INTEGER	NOT NULL	Ошибочных записей
started_at	TIMESTAMP	NOT NULL	Начало
finished_at	TIMESTAMP	NULL	Завершение

IMPORT_ERROR

Ошибки и несопоставленные записи конкретной загрузки.

Поле	Тип	Ключ / ограничение	Назначение
error_id	UUID	PK	Идентификатор ошибки
import_run_id	UUID	FK -> IMPORT_RUN	Запуск загрузки
record_key	VARCHAR(120)		Ключ исходной записи
error_code	VARCHAR(50)	NOT NULL	Код ошибки
message	TEXT	NOT NULL	Описание
raw_payload	JSONB	NULL	Исходные данные для диагностики

1.4 Financial Integration Service - finance_db

SALES_DOCUMENT

Документ реализации продукции.

Поле	Тип	Ключ / ограничение	Назначение
sales_document_id	UUID	PK	Идентификатор документа
external_id	VARCHAR(80)	UQ, NOT NULL	Код финансовой системы
document_date	DATE	NOT NULL	Дата реализации
customer_ref	VARCHAR(80)		Обезличенная ссылка на клиента
currency	CHAR(3)	NOT NULL	Валюта

SALES_ITEM

Позиция документа реализации.

Поле	Тип	Ключ / ограничение	Назначение
sales_item_id	UUID	PK	Идентификатор позиции
sales_document_id	UUID	FK -> SALES_DOCUMENT	Документ реализации
product_id	UUID	NOT NULL	Продукция из Master Data
quantity	DECIMAL(18,4)	NOT NULL	Количество
revenue	DECIMAL(18,2)	NOT NULL	Выручка без налога
discount	DECIMAL(18,2)	NOT NULL	Скидка

FINANCIAL_ENTRY

Накладной расход или корректировка за период.

Поле	Тип	Ключ / ограничение	Назначение
entry_id	UUID	PK	Идентификатор записи
external_id	VARCHAR(80)	UQ, NOT NULL	Идентификатор проводки
period	DATE	NOT NULL	Отчетный период
cost_center_id	UUID	NOT NULL	Центр затрат из Master Data
entry_type	VARCHAR(30)	NOT NULL	OVERHEAD или CORRECTION
amount	DECIMAL(18,2)	NOT NULL	Сумма
currency	CHAR(3)	NOT NULL	Валюта

Поле	Тип	Ключ / ограничение	Назначение
posted_at	TIMESTAMP	NOT NULL	Дата проведения

IMPORT_RUN

Журнал загрузок финансовых данных.

Поле	Тип	Ключ / ограничение	Назначение
import_run_id	UUID	PK	Идентификатор запуска
source_system	VARCHAR(80)	NOT NULL	Источник
period	DATE	NOT NULL	Отчетный период
status	VARCHAR(30)	NOT NULL	Состояние
records_loaded	INTEGER	NOT NULL	Принятые записи
records_failed	INTEGER	NOT NULL	Ошибочные записи
started_at	TIMESTAMP	NOT NULL	Начало
finished_at	TIMESTAMP	NULL	Завершение

IMPORT_ERROR

Ошибки финансовой загрузки.

Поле	Тип	Ключ / ограничение	Назначение
error_id	UUID	PK	Идентификатор ошибки
import_run_id	UUID	FK -> IMPORT_RUN	Запуск загрузки
record_key	VARCHAR(120)		Ключ записи
error_code	VARCHAR(50)	NOT NULL	Код ошибки
message	TEXT	NOT NULL	Описание
raw_payload	JSONB	NULL	Исходная запись

1.5 Costing Service - costing_db

COST_CALCULATION

Версия расчета себестоимости.

Поле	Тип	Ключ / ограничение	Назначение
calculation_id	UUID	PK	Идентификатор расчета
product_id	UUID	NOT NULL	Продукция
period	DATE	NOT NULL	Период
version	INTEGER	NOT NULL	Версия пересчета
status	VARCHAR(30)	NOT NULL	COMPLETED, INVALIDATED
material_cost	DECIMAL(18,2)	NOT NULL	Материальные затраты
labor_cost	DECIMAL(18,2)	NOT NULL	Трудовые затраты
overhead_cost	DECIMAL(18,2)	NOT NULL	Накладные расходы
total_cost	DECIMAL(18,2)	NOT NULL	Полная себестоимость
produced_qty	DECIMAL(18,4)	NOT NULL	Объем выпуска
unit_cost	DECIMAL(18,4)	NOT NULL	Себестоимость единицы
created_at	TIMESTAMP	NOT NULL	Время расчета

OVERHEAD_ALLOCATION

Результат распределения накладных расходов в конкретной версии расчета.

Поле	Тип	Ключ / ограничение	Назначение
allocation_id	UUID	PK	Идентификатор распределения
calculation_id	UUID	FK -> COST_CALCULATION	Версия калькуляции
production_order_id	UUID	NOT NULL	Заказ из Production Service
cost_center_id	UUID	NOT NULL	Центр затрат
basis_type	VARCHAR(40)	NOT NULL	Тип базы распределения

Поле	Тип	Ключ / ограничение	Назначение
basis_value	DECIMAL(18,4)	NOT NULL	Значение базы
amount	DECIMAL(18,2)	NOT NULL	Распределенная сумма

1.6 Profitability Analytics Service - analytics_db

PROFITABILITY_RESULT

Итоговый результат по продукции и периоду.

Поле	Тип	Ключ / ограничение	Назначение
result_id	UUID	PK	Идентификатор результата
calculation_id	UUID	UQ, NOT NULL	Ссылка на версию Costing Service
product_id	UUID	NOT NULL	Продукция
period	DATE	NOT NULL	Отчетный период
revenue	DECIMAL(18,2)	NOT NULL	Выручка
cost_of_sales	DECIMAL(18,2)	NOT NULL	Себестоимость реализованного объема
profit	DECIMAL(18,2)	NOT NULL	Прибыль
margin_percent	DECIMAL(7,2)	NOT NULL	Рентабельность продаж
calculated_at	TIMESTAMP	NOT NULL	Время расчета

PROFITABILITY_AGGREGATE

Агрегированные показатели для группировки и сравнения.

Поле	Тип	Ключ / ограничение	Назначение
aggregate_id	UUID	PK	Идентификатор агрегата
dimension_type	VARCHAR(30)	NOT NULL	PRODUCT_GROUP или COST_CENTER
dimension_id	UUID	NOT NULL	Идентификатор измерения
period	DATE	NOT NULL	Период
revenue	DECIMAL(18,2)	NOT NULL	Выручка
total_cost	DECIMAL(18,2)	NOT NULL	Затраты
profit	DECIMAL(18,2)	NOT NULL	Прибыль
margin_percent	DECIMAL(7,2)	NOT NULL	Рентабельность
updated_at	TIMESTAMP	NOT NULL	Время обновления

1.7 Reporting Service - reporting_db

PROFITABILITY_READ_MODEL

Денормализованная витрина для быстрых GraphQL Query.

Поле	Тип	Ключ / ограничение	Назначение
row_id	UUID	PK	Идентификатор строки
source_result_id	UUID	UQ, NOT NULL	Результат Analytics Service
product_id	UUID	NOT NULL	Продукция
group_id	UUID	NOT NULL	Группа продукции
period	DATE	NOT NULL	Период
revenue	DECIMAL(18,2)	NOT NULL	Выручка
total_cost	DECIMAL(18,2)	NOT NULL	Полная себестоимость
profit	DECIMAL(18,2)	NOT NULL	Прибыль
margin_percent	DECIMAL(7,2)	NOT NULL	Рентабельность
updated_at	TIMESTAMP	NOT NULL	Время обновления

SAVED_REPORT

Сохраненный набор параметров отчета.

Поле	Тип	Ключ / ограничение	Назначение
report_id	UUID	PK	Идентификатор отчета
user_id	UUID	NOT NULL	Владелец из Identity Service
name	VARCHAR(200)	NOT NULL	Название
parameters	JSONB	NOT NULL	Фильтры, группировки и сортировка
created_at	TIMESTAMP	NOT NULL	Создание
updated_at	TIMESTAMP	NOT NULL	Изменение

REPORT_EXPORT

Задание на формирование файла.

Поле	Тип	Ключ / ограничение	Назначение
export_id	UUID	PK	Идентификатор выгрузки
report_id	UUID	FK -> SAVED_REPORT, NULL	Сохраненный отчет
requested_by	UUID	NOT NULL	Пользователь из Identity Service
format	VARCHAR(20)	NOT NULL	XLSX, CSV или PDF
status	VARCHAR(30)	NOT NULL	QUEUED, READY, FAILED
file_ref	VARCHAR(500)	NULL	Ссылка на готовый файл
created_at	TIMESTAMP	NOT NULL	Создание
completed_at	TIMESTAMP	NULL	Завершение
error_message	TEXT	NULL	Ошибка

1.8 Связи внутри баз данных

База	Связи	Комментарий
identity_db	USER 1:N USER_ROLE; ROLE 1:N USER_ROLE	USER_ROLE реализует M:N пользователей и ролей.
master_data_db	PRODUCT_GROUP 1:N PRODUCT; PRODUCT_GROUP 1:N PRODUCT_GROUP; PRODUCT 1:N PRODUCT_MATERIAL; MATERIAL 1:N PRODUCT_MATERIAL	PRODUCT_MATERIAL реализует M:N продукции и материалов.
production_db	PRODUCTION_ORDER 1:N MATERIAL_CONSUMPTION; PRODUCTION_ORDER 1:N LABOR_ENTRY; IMPORT_RUN 1:N IMPORT_ERROR	product_id, material_id и cost_center_id являются внешними идентификаторами, но не FK.
finance_db	SALES_DOCUMENT 1:N SALES_ITEM; IMPORT_RUN 1:N IMPORT_ERROR	product_id и cost_center_id не являются межбазовыми FK.
costing_db	COST_CALCULATION 1:N OVERHEAD_ALLOCATION	Связи с Production и Master Data выполняются по UUID через контракты.
analytics_db	Логическая связь RESULT -> AGGREGATE формируется расчетом, физический FK не нужен	Обе таблицы принадлежат Analytics Service.
reporting_db	SAVED_REPORT 1:N REPORT_EXPORT	Read model обновляется событиями, source_result_id не является FK к analytics_db.

1.9 Дополнения модели и жизненный цикл результатов

Master Data Service хранит утверждённые планы PLANNED_COST: plan_id UUID PK, product_id UUID, period DATE, version INTEGER, unit_cost DECIMAL(18,4), status VARCHAR(30); UNIQUE(product_id, period, version). Источник истины плана — master_data_db, а не фактическая калькуляция. Costing сохраняет planned_unit_cost и версию плана в source_versions.

COST_CALCULATION содержит source_versions JSON с версиями Master Data, Production и Financial. Изменение исходных данных переводит зависимые COMPLETED в INVALIDATED; история не перезаписывается. В синхронной реализации калькуляция сохраняется сразу в COMPLETED. CalculationJob принадлежит Analytics, создаётся после успешного расчёта рентабельности и получает COMPLETED; при инвалидации становится INVALIDATED. calculateCost возвращает список CostBreakdown, calculateProfitability — CalculationJob. Ошибка до завершения возвращается как ошибка операции и не выдаётся за успешное задание.

Financial Integration хранит FINANCIAL_ENTRY с entry_type OVERHEAD/CORRECTION, исходной суммой и cost_center_id. GetOverheadData возвращает нетто-суммы по центрам с учётом корректировок; глобальное correction_amount сохраняется для совместимости и равно нулю. OVERHEAD_ALLOCATION — результат Costing, а не входной финансовый факт.

production_db.IMPORT_RUN и financial_db.IMPORT_RUN — разные таблицы с отдельными владельцами и локальными IMPORT_ERROR. В интеграционных базах и costing_db предусмотрен OUTBOX_EVENT: event_id UUID PK, envelope JSON, published_at TIMESTAMP NULL, last_error TEXT NULL. Это позволяет восстановить доставку уже сохранённого бизнес-факта.

2 GraphQL API

Gateway принимает пользовательский JWT, проверяет сессию через Auth Service и применяет роли. Внутренние бизнес-операции выполняются через служебный JWT и gRPC. Публичная регистрация даёт только PENDING; аналитические полномочия назначает оператор. Производственный бухгалтер видит только ERP/MES, финансовый — FINANCE, аналитик и администратор — оба контура.

```
enum CalculationStatus { COMPLETED INVALIDATED }
input ProfitabilityFilterInput {
  productIds: [ID!]
  groupIds: [ID!]
  costCenterIds: [ID!]
  periodFrom: Date!
  periodTo: Date!
  minMarginPercent: Decimal
}
type CostBreakdown {
  calculationId: ID!
  productId: ID!
  period: Date!
  version: Int!
  materialCost: Decimal!
  laborCost: Decimal!
  overheadCost: Decimal!
  totalCost: Decimal!
  producedQty: Decimal!
  unitCost: Decimal!
  plannedUnitCost: Decimal
  unitCostDeviation: Decimal
}
```

```
type OverheadAllocation {
  calculationId: ID!
  productionOrderId: ID!
  costCenterId: ID!
  basisType: String!
  basisValue: Decimal!
  amount: Decimal!
}

type Query {
  products(filter: ProductFilterInput, sort: ProductSortInput): [Product!]!
  importRuns(source: ImportSource, period: Date): [ImportRun!]!
  costBreakdown(productId: ID!, period: Date!, version: Int): CostBreakdown
  overheadAllocations(productId: ID!, period: Date!, version: Int,
    costCenterId: ID): [OverheadAllocation!]!
  profitability(filter: ProfitabilityFilterInput!,
    sort: ProfitabilitySortInput): [ProfitabilityResult!]!
  profitabilitySummary(filter: ProfitabilityFilterInput!): ProfitabilitySummary!
  calculationStatus(id: ID!): CalculationJob
  reportExport(id: ID!): ReportExport
}
type Mutation {
  startProductionImport(input: StartImportInput!): ImportRun!
  startFinancialImport(input: StartImportInput!): ImportRun!
  retryImport(id: ID!): ImportRun!
  resolveExternalMapping(input: ResolveMappingInput!): ExternalMapping!
  calculateCost(input: CostCalculationInput!): [CostBreakdown!]!
  calculateProfitability(input: ProfitabilityCalculationInput!): CalculationJob!
  exportReport(input: ExportReportInput!): ReportExport!
}
```

Отсутствующий productIds означает отсутствие ограничения, пустой [] означает отсутствие подходящих продуктов. Фильтры продукции, групп и центров затрат пересекаются. costCenterIds выбирает продукты, участвующие в распределении выбранного центра; overheadAllocations раскрывает суммы и базу каждого заказа. profitability и profitabilitySummary используют одинаковые фильтры и Reporting read model. Это сохраняет возможности плана/факта и анализа центров затрат из ЛР1.

3 Типизированные gRPC контракты

Идентификаторы UUID и даты ISO передаются строками; DECIMAL также передаётся строкой. Источник контрактов — docs/proto/profitability.proto. Метод LookupExternalMapping используется интеграционными сервисами до сохранения записи. Запрос не изменяет сопоставления; изменение выполняется отдельной защищённой операцией.

```
service MasterDataService {
  rpc LookupExternalMapping(LookupExternalMappingRequest)
      returns (LookupExternalMappingResponse);
  rpc GetCostingContext(GetCostingContextRequest)
      returns (GetCostingContextResponse);
}
message LookupExternalMappingRequest {
  string source_system = 1;
  string entity_type = 2; // PRODUCT, MATERIAL, COST_CENTER
  string external_code = 3;
}
message LookupExternalMappingResponse {
  string internal_id = 1;
  string mapping_version = 2;
}
message ProductionAllocationBasis {
  string production_order_id = 1;
  string cost_center_id = 2;
  string volume = 3;
```

```
  string labor_hours = 4;
  string direct_cost = 5;
}
message GetProductionCostDataResponse {
  string produced_qty = 1;
  string material_cost = 2;
  string labor_cost = 3;
  repeated string production_order_ids = 4;
  string data_version = 5;
  string labor_hours = 6;
  repeated ProductionAllocationBasis allocation_lines = 7;
}
```

basis_value при записи OVERHEAD_ALLOCATION выбирается из volume, labor_hours или direct_cost согласно allocationBase. Детализация несёт production_order_id и cost_center_id, поэтому распределение выполняется по каждому центру отдельно и замыкается на реальные производственные заказы.

```
service FinancialDataService {
  rpc GetOverheadData(GetOverheadDataRequest)
      returns (GetOverheadDataResponse);
  rpc GetSalesData(GetSalesDataRequest) returns (GetSalesDataResponse);
}
message GetOverheadDataRequest {
  string period = 1;
  repeated string cost_center_ids = 2;
}
message OverheadAmount {
  string cost_center_id = 1;
  string amount = 2; // OVERHEAD + CORRECTION for this center
}
message GetOverheadDataResponse {
  repeated OverheadAmount items = 1;
  string correction_amount = 2; // 0; corrections already included
  string data_version = 3;
}
service CostingService {
  rpc GetApprovedCostCalculation(GetApprovedCostCalculationRequest)
      returns (GetApprovedCostCalculationResponse);
}
message GetApprovedCostCalculationRequest {
  string product_id = 1;
  string period = 2;
  optional int32 version = 3;
}
```

GetApprovedCostCalculation возвращает calculation_id, version, unit_cost, total_cost, produced_qty и status. Для INVALIDATED возвращается FAILED_PRECONDITION. При обработке CostCalculationCompleted параметр version обязателен по сценарию: Analytics передаёт именно версию из payload. Для ручного расчёта без версии выбирается последняя пригодная калькуляция. GetSalesData возвращает sold_qty, revenue, currency и data_version; GetProfitabilityDetail — result_id, revenue, cost_of_sales, profit, margin_percent, calculated_at.

План получается через MasterDataGateway.GetPlannedCost(product_id, period): plan_id, version, unit_cost и status=APPROVED либо null. JSON bridge-контракт дополнительно фиксирует ListCostCenterProducts, ListOverheadAllocations и запросы Reporting. Он используется Gateway, а межсервисные данные производства, финансов и калькуляции передаются типизированными сообщениями.

4 Kafka события и реакции

```
{
  "eventId": "UUID",
  "eventType": "CostCalculationCompleted",
  "occurredAt": "2026-10-31T12:00:00Z",
  "producer": "costing-service",
  "schemaVersion": 1,
  "correlationId": "UUID одного бизнес-сценария",
  "payload": {
    "calculationId": "UUID", "productId": "UUID",
    "period": "2026-09-01", "version": 2,
    "unitCost": "40.4000", "totalCost": "4040.00",
    "sourceVersions": {
      "master": "hash нормативов", "production": "ревизия импорта",
      "financial": "ревизия импорта", "plan": 1
    }
  }
}
```

Событие	Payload и потребители
ProductionDataImported	importRunId, sourceSystem, period, recordsLoaded, dataVersion. Costing обновляет готовность и инвалидирует прежние расчёты; Analytics и Reporting снимают старые результаты.
FinancialDataImported	Те же идентификаторы финансовой загрузки. Реакции Costing, Analytics и Reporting аналогичны.
ImportFailed	Реальный importRunId, sourceSystem, period, errorCount, primaryErrorCode. Costing сбрасывает готовность контура; зависимые результаты инвалидируются.
MasterDataChanged	entityType, entityId, changeType, mappingVersion. Costing очищает кеш и помечает калькуляции INVALIDATED; Analytics и Reporting исключают устаревшие данные.
CostCalculationCompleted	Версия и идентификатор расчёта, productId, period, суммы, sourceVersions. Analytics рассчитывает точную версию; Reporting ожидает её проекцию.
ProfitabilityCalculated	resultId, productId, groupId, period, revenue, costOfSales, profit, marginPercent, costVersion. Reporting обновляет витрину; correlationId унаследован от исходного события.

Успех импорта публикуется только после сохранения записей. Состояние загрузки и его событие сохраняются через локальный outbox. Для Costing одной транзакцией фиксируются калькуляция, OVERHEAD_ALLOCATION и событие. Подтверждение Kafka offset разрешено только после успешной обработки либо подтверждённой передачи в DLQ; ошибки декодирования обрабатываются внутри цикла записей.

5 Сценарии взаимодействия

Сценарий	Последовательность
Импорт	Проверка роли и источника → создание локального IMPORT_RUN → чтение JSON-снимка → LookupExternalMapping → транзакционное сохранение записи → счётчики/ошибки → COMPLETED либо FAILED → outbox → Kafka.
Повтор импорта	Проверка FAILED и полномочий контура → очистка прошлых ошибок → повторное чтение исправленного снимка → upsert по source/external_id без дубликатов → итоговое состояние.
Калькуляция	Нормативы и план → факты по заказам/центрам → исходные накладные суммы → проверка базы → распределение с остатком округления → сохранение версий и деталей → outbox.
Рентабельность	CostCalculationCompleted → чтение указанной версии → получение реализации → расчёт стоимости продаж и прибыли → результат и CalculationJob COMPLETED → ProfitabilityCalculated с прежним correlationId.
Просмотр	JWT и роль → пересечение фильтров → Reporting.ListProfitability/ProfitabilitySummary → текущая read model, без SQL к чужой базе.
Экспорт	Проверка владельца SavedReport до создания задания → чтение параметров и витрины → запись временного CSV/XLSX/PDF → атомарное переименование → file_ref и READY; при ошибке FAILED.

6 Архитектура и последовательность

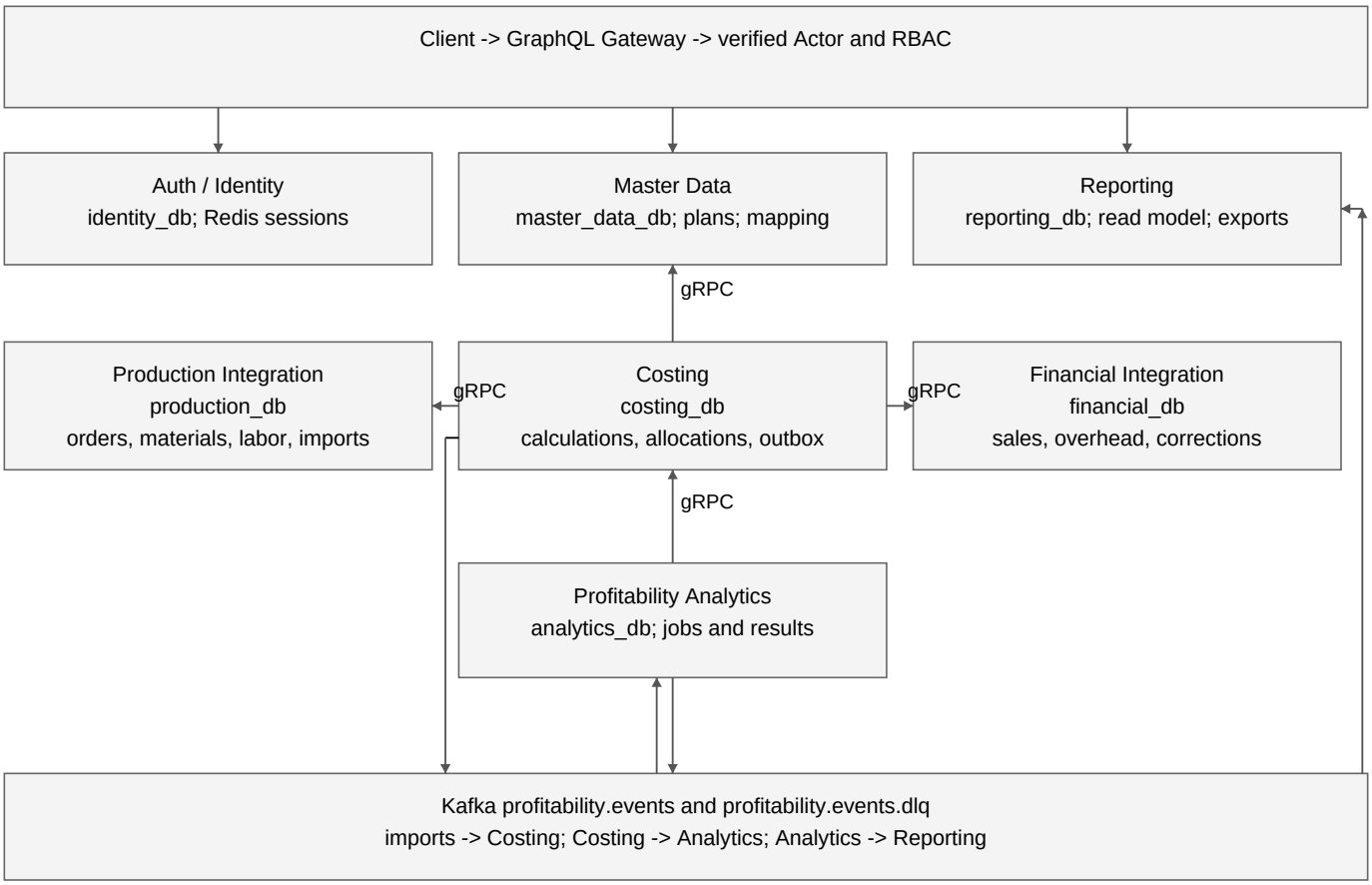


Рисунок. Сервисы, отдельные базы данных и асинхронное обновление Reporting

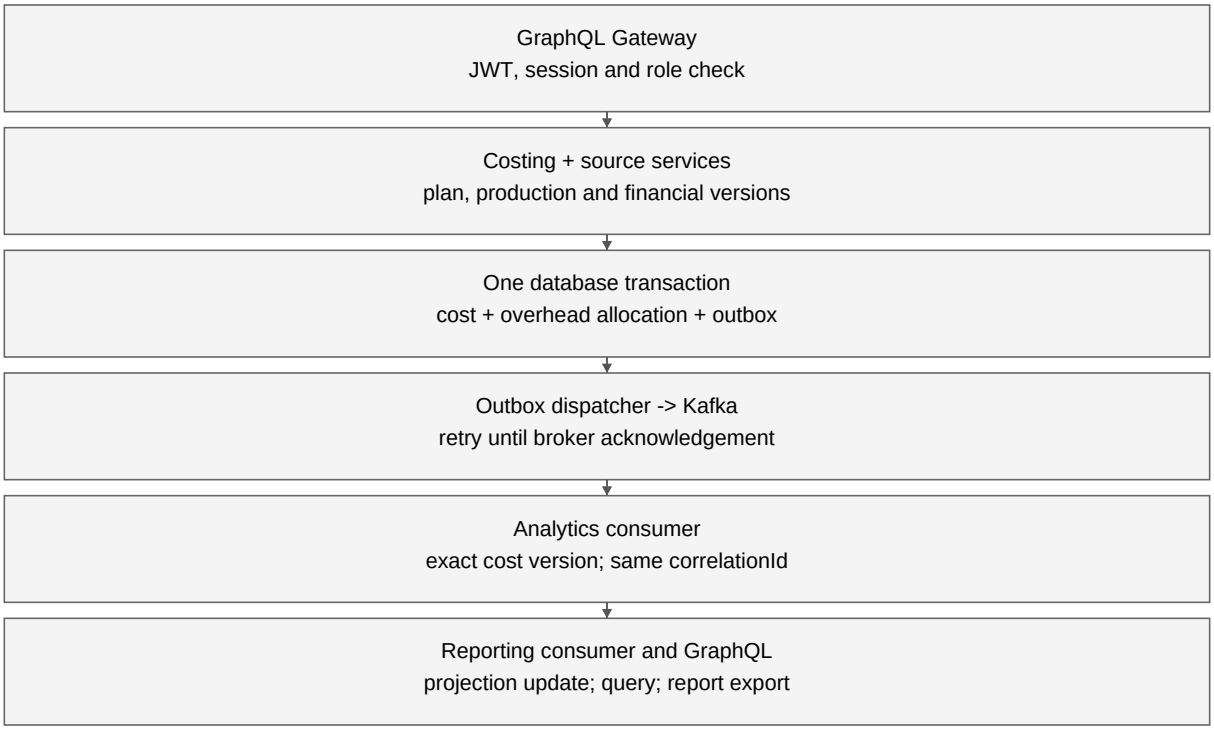


Рисунок. Последовательность расчёта и получения результатов

7 Планирование и проверка контрактов

Реализация развивается последовательно: структура сервисов и Query в ЛР3, бизнес-операции в ЛР4, безопасность в ЛР5, сетевые gRPC и Kafka в ЛР6, устойчивость и кеширование в ЛР7. Итоговая исполняемая версия собрана в lab7_profitability_system. Контракты, SQL и события ЛР2 согласуются с этой версией; запуск рабочего приложения в рамках проектирования ЛР2 не требуется.

Проверяются наличие LookupExternalMapping, полнота базы распределения до уровня заказа/центра, сохранение source_versions, nullable-план и отклонение, согласованные состояния COMPLETED/INVALIDATED, одинаковое поведение пустого фильтра в списке и сводке, сохранение correlationId и восстановление доставки события после отказа Kafka.

Заключение

Технический проект замыкает импорт, калькуляцию и аналитическую выдачу едиными контрактами. Источники и версии исходных данных сохраняются вместе с расчётом, план и внешние сопоставления имеют владельца, распределение накладных расходов детализируется по заказам и центрам. Состояния объектов и правила обработки событий согласованы с итоговой реализацией.